

Project 3: Full Stack Engineering

Project Title: Real-Time Collaboration Platform

Duration: 20 May 2026 – 9 June 2026

Deadline: 9 June 2026

1. Project Overview

Modern teams rely on tools like Slack, Notion, and Microsoft Teams to communicate and collaborate in real time. These platforms allow multiple users to interact simultaneously, share updates instantly, and work together on shared content.

In this project, you will build a **real-time collaboration platform** that enables users to communicate, manage shared workspaces, and edit content together live.

This project combines backend APIs, frontend development, and real-time communication using WebSockets.

2. Problem Statement

Teams need a system where:

- Users can communicate instantly
- Multiple people can work within shared workspaces
- Updates are reflected in real time without refreshing the page
- Collaboration is smooth and interactive

Your task is to build a platform that provides these capabilities using a full stack architecture.

3. Project Objectives

By completing this project, you should be able to:

- Build and integrate a full stack application
- Implement real-time communication using WebSockets
- Design user-friendly interfaces
- Manage state between frontend and backend

- Handle asynchronous events and live updates

4. What You Need to Build

You are required to build a **full stack collaboration system** consisting of:

- Backend APIs and real-time services
- Frontend user interface
- Integration between both layers

The system should allow multiple users to interact within shared workspaces in real time.

5. Functional Requirements

Your platform must include the following features:

5.1 User Authentication

- Users should be able to:
 - Sign up
 - Log in
 - Log out
- Authentication should be secure (token-based)
- Protected routes should require login

5.2 Workspace Management

- Users can:
 - Create workspaces
 - Join existing workspaces
- Each workspace should act as a separate collaboration environment

5.3 Real-Time Chat System

- Users within a workspace can:
 - Send messages instantly
 - Receive messages in real time (without page refresh)
- Messages should include:
 - Sender
 - Timestamp

- Content

5.4 Shared Notes Editor

- Users should be able to:
 - Create shared notes
 - Edit notes collaboratively
- Updates should reflect in real time for all users in the workspace

5.5 Notifications System

- Users should receive notifications for:
 - New messages
 - Workspace activity
 - Mentions (optional)
- Notifications can be:
 - Real-time alerts
 - UI-based indicators

6. Technical Requirements

You must use the following technologies:

Backend

- Django
- Django REST Framework for APIs
- WebSockets for real-time communication (Django Channels recommended)

Frontend

- Next.js
- Component-based UI
- API integration with backend

Real-Time Communication

- WebSocket implementation is mandatory
- Must handle:
 - Live messaging

- Real-time updates

7. Suggested System Architecture (Guidance)

You may structure your system as follows:

Backend

- Authentication module
- Workspace module
- Chat module (WebSockets)
- Notes module

Frontend

- Login/Signup pages
- Dashboard
- Workspace UI
- Chat interface
- Notes editor

8. Advanced Requirements

To achieve a higher evaluation score, implement the following:

8.1 Online/Offline User Presence

- Show which users are currently online
- Update status in real time

8.2 Version History of Notes

- Track changes in shared notes
- Allow users to view previous versions

8.3 File Sharing

- Allow users to:
 - Upload files
 - Share files within workspace
- Files should be accessible to workspace members

9. Deliverables

By the deadline, you must submit:

1. Fully working frontend and backend system
2. Live deployed application (strongly recommended)
3. Demo video showing:
 - Real-time chat
 - Workspace usage
 - Notes collaboration
4. GitHub repository with:
 - Clean and structured code
 - Proper commit history
 - README file

10. README Requirements

Your README must include:

- Project overview
- Architecture explanation
- Tech stack used
- Setup instructions
- Screenshots of UI
- Explanation of real-time implementation

11. Evaluation Criteria

Your project will be evaluated based on:

11.1 Real-Time System Performance

- Are updates truly real time?
- Is communication smooth and reliable?

11.2 UI/UX Usability

- Is the interface clean and easy to use?
- Can users navigate easily?

11.3 Integration Quality

- Is frontend properly connected to backend?
- Are APIs used correctly?

11.4 Code Quality

- Clean, modular, and maintainable code
- Proper separation of concerns

11.5 Feature Completion

- All required features implemented correctly

12. Important Instructions

- Do not fake real-time behavior using page refresh

- Use proper WebSocket implementation
- Focus on both functionality and user experience
- Test with multiple users simultaneously
- Keep your UI simple but effective